НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла):
студент групи АІ-241
Швець Д.Я.
Перевірив:
ас. Баранов О.М.

Одеса – 2026

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт /api/generate. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	248	5
Час, с	2,15	0,12
Токенів/с	115,35	41,67

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	1 / 5
eval_count	412	28
Час, с	3,65	0,68

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальної моделі?

Відповідь: Додаткові токени пішли на прихований ланцюжок міркувань (блок thinking): перед остаточною відповіддю модель розписує кроки та перевіряє себе. На фактичному питанні це дало 248 токенів проти 5 в інструктивної моделі, на задачі – 412 проти 28.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Перевагу міркувальні моделі дають на багатокрокових задачах: арифметичних і логічних задачах із «пасткою», виведенні алгоритмів, головоломках. На простих фактичних запитах і переказі їх використання недоцільне.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Міркувальну модель додатково навчають із підкріпленням (RL) на траєкторіях міркувань (Chain-of-Thought): винагороджується правильний кінцевий результат, тож модель вчиться будувати й перевіряти хід розв’язання. Інструктивну модель навчають переважно дотримуватися інструкцій на прикладах «запит – відповідь».

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні. Кількість токенів – це міра обсягу виконаної роботи, а не якості. На питанні про столицю 248 токенів не дали жодної переваги над 5 токенами, а на задачі про ручку й зошит додаткові токени окупилися лише тому, що пішли на корисні кроки розв’язання.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Швидкість генерації (токенів/с) залежить від архітектури, кількості параметрів, квантування та обладнання. Крім того, на дуже короткій відповіді (5 токенів) фіксовані витрати на обробку запиту займають більшу частку часу, що занижує показник токенів/с, тому порівнювати моделі лише за ним некоректно.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 17,9 разів. На задачі про ручку й зошит ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для

агента з 8 послідовними викликами різниця складе 29,2 с проти 5,4 с, що робить міркувальну модель непридатною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Тестове питання Варіанта 2: «Поясни різницю між стеком і чергою.»

Таблиця 3 – Вплив system-повідомлення

Модель	System	Довжина, симв.	Мова	Речень	Виконано?
llama3.2:3b	лаконічний асистент	320	Українська	4	Ні
llama3.2:3b	українською, 2 речення	145	Українська	2	Так
qwen3:4b	лаконічний асистент	680	Українська	7	Ні
qwen3:4b	українською, 2 речення	162	Українська	2	Так

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	28	62	90
llama3.2:3b	українською, 2 речення	32	31	63
qwen3:4b	лаконічний асистент	26	310	336
qwen3:4b	українською, 2 речення	30	185	215

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Обидві моделі дотрималися обмеження у 2 речення при явній інструкції, проте llama3.2:3b відповіла лаконічніше та прямолінійніше, без зайвих вступних слів.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: При прямому вказуванні мови в системному промпті обидві моделі відповідали українською. Однак при абстрактному промпті малі моделі іноді збиваються на кальковані вирази.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: Показник prompt_tokens збільшився на 4 токени в обох моделей, оскільки нова системна інструкція містила більше слів та символів, ніж базова.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: У міркувальної моделі qwen3:4b показник completion_tokens враховує не лише видиму відповідь, а й усі згенеровані приховані токени роздумів у блоці thinking.

Відповіді на питання до завдання 2 (крок 7)

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови? Це залежить від специфіки датасету навчання та налаштувань вирівнювання (alignment). Модель llama3.2:3b краще відлічує межі речень, тоді як qwen3:4b схильна розгортати міркування навіть при жорстких обмеженнях.

2. Чим system-повідомлення відрізняється від user-повідомлення? system-повідомлення задає системний контекст і глобальну поведінку моделі в системних тегах токенізатора. На практиці воно має вищий пріоритет, проте мала модель може ігнорувати його за наявності суперечностей у user-промпті.

3. Чому міркувальна модель витрачає більше вихідних токенів при однаковій видимій відповіді? Тому що completion_tokens враховує як видимий текст у content, так і приховані токени міркування в блоці thinking.

4. Що станеться при суперечливих інструкціях у system і user? Поведінка стає недетермінованою: більші моделі надають пріоритет system, тоді як малі моделі зазвичай слідують останній інструкції з user.

5. Чому малі моделі гірше дотримуються інструкцій щодо мови? Малі моделі мають обмежений словниковий запас та меншу кількість параметрів для збереження багатомовних граматичних паттернів, через що частіше збиваються на домінуючу англійську мову.

Висновок за завданням 2

Загальне формулювання «лаконічний асистент» не змусило жодну з моделей відповісти коротко (4 і 7 речень), тоді як чітка інструкція з перевірюваною вимогою («українською, 2 речення») була виконана обома моделями. Додаткові токени в system-повідомленні окупаються: completion_tokens зменшилися з 62 до 31 у llama3.2:3b та з 310 до 185 у qwen3:4b, а total_tokens – на 30 % і 36 % відповідно.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна (llama3.2:3b)	Хмарна (llama-3.3-70b-versatile)
Назва моделі	llama3.2:3b	llama-3.3-70b-versatile
Час відповіді, с	0,85	0,42
completion_tokens	45	58
Довжина відповіді, символів	210	280
Мова відповіді	Українська	Українська
Фактичних помилок у відповіді	1 (галюцинація)	0

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити? Лише 2 рядки — base_url та api_key (а також поле model).

2. Яка відповідь повніша й точніша? Хмарна модель 70B надала більш точне порівняння за допомогою аналогії (LIFO vs FIFO), тоді як локальна модель обмежилася сухим академічним визначенням.

3. Чи були у локальної моделі помилки або проблеми з мовою? Локальна модель надала правильне визначення, але сформулювала його більш кострубато.

4. За рахунок чого хмарна модель відповіла швидше? Хмарний сервіс (наприклад, Groq) використовує спеціалізовані прискорювачі (LPU/GPU) з величезною пропускною здатністю пам’яті, що перекриває затримку мережевого запиту.

5. Відповіді на питання до чат-бота:

- Стандарт OpenAI: формат OpenAI став де-факто стандартом завдяки першопрохідництву ринку, що спростило інтеграцію інструментів і дозволило розробникам змінювати провайдерів без переписування коду.
- Вплив розміру: розмір параметрів (70B vs 3B) прямо визначає глибину знань, розуміння контексту та якість дотримання складних інструкцій.
- Конфіденційність та Rate Limits: хмарні API створюють ризик витоку даних, тому корпоративні клієнти укладають B2B-угоди без навчання на даних або розгортають локальні екземпляри. Безкоштовні ліміти (Rate Limits) обходять ротацією ключів, чергами запитів та кешуванням.
-

Відповіді на питання (крок 6)

1.Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama? Стандарт OpenAI став де-факто стандартом завдяки першопрохідництву ринку, що спростило інтеграцію інструментів і дозволило розробникам змінювати провайдерів без переписування коду.

2.Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими

факторами? Розмір параметрів (70B vs 3B) прямо визначає глибину знань, розуміння контексту та якість дотримання складних інструкцій.

3.Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється? Це пояснюється величезною обчислювальною потужністю хмарних серверів, де паралелізм обчислень тензорів набагато ефективніший, ніж на локальному ПК.

4.Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах? Хмарні API створюють ризик витоку даних, тому корпоративні клієнти укладають B2B-угоди без навчання на даних або розгортають локальні екземпляри.

5.Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження? Безкоштовні ліміти (Rate Limits) обмежують кількість запитів на хвилину. Їх обходять ротацією ключів, чергами запитів та кешуванням.

Висновок за завданням 3

Обидві моделі викликаються через однаковий інтерфейс, тому перехід між ними не потребує змін у логіці програми. Хмарна модель відповіла приблизно вдвічі швидше (0,42 с проти 0,85 с), дала розгорнутішу відповідь (58 проти 45 токенів) і не припустилася фактичних помилок, тоді як мала локальна модель видала одну галюцинацію.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 6 – Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	0,120	0,250	85,0
локальна	llama3.2:3b	середній	0,135	0,950	180,2
локальна	llama3.2:3b	довгий	0,140	3,800	210,5
хмарна	llama-3.3-70b	короткий	0,280	0,350	120,0
хмарна	llama-3.3-70b	середній	0,290	0,720	450,0
хмарна	llama-3.3-70b	довгий	0,310	1,850	620,0

Таблиця 7 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	2,850	0,120

Відповіді на питання (крок 6)

1. У якої моделі кращий TTFT? У локальної моделі TTFT кращий на коротких запитах, оскільки відсутня мережева затримка (RTT).

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за симв./с»? Ні. Локальна модель випереджає за TTFT, але хмарна модель має в рази вищу швидкість генерації (симв./с).

3. Як змінилося співвідношення при переході до довгого промпту? На довшому промпті хмарна модель повністю випереджає локальну за загальним часом за рахунок високої швидкості генерації.

4. Чому холодний старт повільніший за теплий? Під час холодного старту виконується зчитування вагових коефіцієнтів з диска в оперативну/відеопам’ять.

Відповіді на питання (крок 7)

1. Що фізично відбувається під час TTFT і чому його вимірюють окремо? Фізично це етап обробки промпту (Prefill): токенізація, паралельне обчислення контексту та створення KV-кешу. Вимірюють окремо, оскільки цей час визначає швидкість відгуку інтерфейсу — як швидко користувач побачить перше слово.

2. Чому хмарна модель має гірший TTFT, але кращу швидкість генерації? TTFT гірший через затримку інтернет-мережі (ping) при відправці запиту. Але швидкість генерації значно вища, бо хмарні сервери використовують потужні промислові GPU/LPU, які обчислюють наступні токени швидше за локальний ПК.

3. Чому перший запуск локальної моделі повільніший (холодний старт)? Холодний старт — це первинне копіювання гігабайтів файлу моделі з повільного диска (SSD) в оперативну або відеопам'ять. Наступні запуски (теплі) швидкі, бо модель вже завантажена в RAM/VRAM.

4. Чому беруть медіану кількох запусків? Щоб відсікти випадкові сплески (outliers), спричинені фоновим навантаженням ОС, троттлінгом процесора або нестабільністю інтернет-з'єднання, та отримати об'єктивну базову швидкість.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді? Обробка запиту (Prefill) виконується паралельно: відеокарта обробляє всі вхідні токени одночасно за один прохід. Генерація відповіді (Decode) є послідовною: кожен наступний токен обчислюється лише після попереднього, що займає більше часу.

Висновок за завданням 4

TTFT майже не залежить від довжини промпту (0,120–0,140 с у локальної та 0,280–0,310 с у хмарної моделі), тоді як загальний час зростає разом з обсягом відповіді. Локальна модель стартує швидше за рахунок відсутності мережевої затримки, але хмарна генерує текст значно швидше (до 620 симв./с проти 210,5). Холодний старт локальної моделі збільшує TTFT до 2,850 с, тому першу відповідь слід вимірювати окремо від наступних.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 8 – Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель

1. Факт (рік відкриття метро — 1960)	2	2	2
2. Формат (JSON)	1	1	2
3. Стиснення (збереження чисел)	1	2	2
4. Код (total_weight)	2	2	2
5. Логіка (рейси вантажівки: 120 ящиків)	0	2	2
6. Українська мова	1	2	2
Разом /12	7	11	12

Таблиця 9 – Розбіжність оцінок між двома прогонами (Так/Ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	Ні	Ні	Ні
2. Формат	Так	Ні	Ні
3. Стиснення	Ні	Ні	Ні
4. Код	Ні	Ні	Ні
5. Логіка	Так	Ні	Ні
6. Українська	Ні	Ні	Ні

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця? Хмарна модель набрала 12 з 12. Основна різниця виникла в дотриманні формату JSON, здатності до логічного міркування та якості української мови.
2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі? У llama3.2:3b оцінки розійшлися у форматі та логіці, що свідчить про низьку надійність малих моделей без структурної валідації.
3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2? Жодна з локальних моделей не виконала її бездоганно, обгортаючи результат у теги, і лише хмарна модель видала сирий JSON.
4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову? Так, найменша модель (llama3.2:3b) періодично вживала кальки та русизми.

Аналіз провалу (крок 7)

Завдання 5 (логіка): 120 ящиків: 45 по 8 кг, решта (75) по 12 кг. Вантажівка розрахована на 1200 кг.

Запит: «Є 120 ящиків: 45 по 8 кг, решта по 12 кг. Вантажівка тримає 1200 кг. Скільки рейсів потрібно? Покажи хід розв'язання.»

Відповідь llama3.2:3b: «Загальна вага: 45*8 + 75*12 = 360 + 900 = 1260 кг. Вантажівка бере 1200 кг. Отже потрібен 1 рейс.»

Правильна відповідь: загальна вага 1260 кг > 1200 кг, тому потрібно 2 рейси.

Пояснення: модель правильно обчислила суму (1260), але зробила помилку в логічному порівнянні чисел, не врахувавши округлення кількості рейсів у більшу сторону: 1260 / 1200 = 1,05 → 2.

Відповіді на питання (крок 8)

1. Які ще підходи до оцінювання використовують на практиці? Автоматизовані бенчмарки (MMLU, HumanEval, GSM8K), сліпе A/B-тестування людьми (наприклад, Chatbot Arena) та метод LLM-as-a-Judge (коли потужніша модель, як-от GPT-4, оцінює відповіді менших моделей за заданими критеріями).

2. Чому мовні моделі ігнорують інструкцію не використовувати markdown для JSON? У гігантських масивах навчальних даних (як-от код із GitHub) блоки JSON майже завжди обгорнуті в markdown (```json). Цей завчений візуальний патерн настільки сильний, що часто пересилює пряму текстову заборону в промпті.

3. Чому малі моделі гірше пишуть українською, ніж англійською? Частка українських текстів у навчальних датасетах зазвичай становить менше 1% (проти 80-90% для англійської). Малим моделям фізично не вистачає параметрів, щоб глибоко засвоїти словник і граматику «малоресурсних» мов.

4. Чому та сама модель дає різні відповіді при повторному запуску? Через імовірнісну природу генерації (якщо параметр temperature більший за 0). Замість того, щоб завжди обирати одне "найкраще" слово, модель щоразу заново обирає (семплює) наступний токен із певного розподілу ймовірностей.

5. Що таке галюцинація і чому модель вигадує факт? Галюцинація — це генерація граматично правильної та переконливої, але повністю хибної інформації. Модель не містить бази даних фактів і не має концепції «я не знаю». Її єдина мета — математично продовжувати текст найбільш імовірними словами, тому вона заповнює прогалини в знаннях правдоподібними вигадками.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 10 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	5
temperature	0.7	4	5
temperature	1.5	5	1
top_p	0.1	1	5
top_p	0.5	3	5
top_p	1.0	5	3

Творче завдання: назва застосунку нагадувань про ліки. Фактичне завдання: рік відкриття Київського метрополітену (1960).

Таблиця 11 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (Так/Ні)	Так

Відповіді на питання (крок 8)

1.Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена? Він ділить значення логітів на коефіцієнт T перед проходженням функції Softmax.

2.Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом? Top_p (nucleus sampling) сортує токени за спаданням ймовірності і залишає лише ті, чия кумулятивна сума досягає P, відкидаючи інші.

3.Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне? Рекомендується налаштовувати лише один із них, залишаючи інший за замовчуванням (наприклад, змінювати лише температуру).

4.Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках? Бо паралельні тензорні операції на сучасних GPU можуть призводити до мікроскопічних розбіжностей при округленні чисел із плаваючою комою.

5.Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту? Для структурованих даних використовують 0.0 - 0.2, для творчого тексту — 0.8 - 1.2.

Завдання 7. Підсумкове порівняння моделей

Таблиця 12 – Підсумкове порівняння моделей

Критерій	Локальна мала (llama3.2:3b)	Локальна більша (qwen3:4b)	Хмарна (llama-3.3-70b-versatile)
Швидкість генерації	Середня (~85 симв./с)	Низька (~40 симв./с з CoT)	Дуже висока (до 620 симв./с)
TTFT	Відмінний (~0,12 с)	Добрий (~0,25 с)	Посередній (~0,30 с)
Якість, балів /12	7/12	11/12	12/12
Стабільність формату	Низька (додає markdown)	Середня	Висока
Потрібні ресурси	8 ГБ RAM	16 ГБ RAM	Інтернет + API ключ
Вартість	Безкоштовно	Безкоштовно	Безкоштовно (з лімітами)
Куди йдуть дані	Залишаються локально	Залишаються локально	Передаються провайдеру
Працює без інтернету	Так	Так	Ні
Коли обирати її	Для простих задач на пристроях з обмеженою RAM та вимогами до приватності.	Для складної логіки та аналізу, коли важлива автономність і приватність.	Для продакшену, обробки великих задач та суворого витримування формату.

Висновок

У ході виконання лабораторної роботи було розгорнуто середовище Ollama та проведено порівняльний аналіз локальних та хмарних мовних моделей. Досліджено використання уніфікованого інтерфейсу OpenAI API для легкої зміни провайдера. Емпірично доведено, що міркувальні моделі вимагають більше ресурсів через внутрішній генеративний процес CoT, а значення temperature=0.0 є критичним для забезпечення стабільності відповіді та точності формату даних у JSON.